

Article Title

Lucas Mello Schnorr¹, Somebody Else²

¹ Institute of Informatics, Federal University of Rio Grande do Sul (UFRGS)
Caixa Postal 15.064 – 91.501-970 – Porto Alegre – RS – Brazil

²Somebody else's Institution, Address – Brazil

schnorr@inf.ufrgs.br

***Abstract.** Put the abstract here.*

***Resumo.** Coloque o resumo aqui.*

1. Introduction

This is just an example to show how Orgmode [Schulte et al. 2012] can be neatly used to write papers following the SBC Article latex style. Feel free to modify and distributed this as you wish.

Be aware that this example is compatible with OrgMode 9.*. Earlier or later OrgMode versions require minor modifications to the Makefile and/or ieee.org files.

For a detailed explanation on why and how to write reproducible articles using OrgMode, please visit the following page.

2. Apresentação do Algoritmo

2.1. Explicação do Algoritmo

O algoritmo Q-Learning é um método de aprendizado por reforço no qual um agente aprende a alcançar um objetivo final utilizando uma tabela chamada Q-Table. Essa tabela armazena valores para cada par de estado e ação, indicando o quão boa é determinada ação em um estado específico. Com base nesses valores, o agente tende a escolher as ações que possuem as maiores recompensas esperadas.

Para construir essa tabela, o agente precisa passar por um processo de treinamento composto por vários episódios. Em cada episódio, o agente executa uma sequência de passos (steps). A cada passo, uma ação é escolhida, o ambiente retorna uma recompensa, e o valor correspondente na Q-Table é atualizado. Ao longo do treinamento, o agente aprende quais ações produzem os melhores resultados para alcançar o objetivo desejado.

2.2. Problema Escolhido

O algoritmo Q-Learning foi escolhido para resolver um problema de pathfinding em um mundo em formato de grade. O desafio consiste em fazer um agente chegar ao ponto final utilizando apenas quatro ações de movimento: mover-se para cima, para baixo, para a esquerda e para a direita.

2.3. Pseudocódigo

```
1 ALGORITMO: Q-Learning Sequencial
2 -----
3 ENTRADAS:
4     Grid(L, C), Obstáculos(O), Episódios(E), Max_Passos(P)
5     alfa (Aprendizado), gama (Desconto), epsilon (Exploração)
6
7 1. INICIALIZAÇÃO E AMBIENTE
8     |   Alocar dinamicamente Q_Table[Estados][Ações]
9     |   Preencher Q_Table com zeros
10    |   Gerar Grid(L, C) com:
11    |       - Estado_Inicial s0 em (0, 0)
12    |       - Estado_Objetivo g em (L-1, C-1)
13    |       - Posicionar O obstáculos aleatórios (evitando s0 e g)
14    |   Configurar Recompensas:
15    |       - R_objetivo (+100)
16    |       - R_obstáculo (-100)
17    |       - R_passo (-1)
18    |       - R_loop (-10)
19
20 2. NUCLEO DE APRENDIZADO (Treinamento)
21    |   PARA cada episódio de 1 até E:
22    |       |   s <- s0 (Estado Inicial)
23    |       |   PARA cada passo de 1 até P:
24    |       |       |
25    |       |       |   // Seleção de Ação (epsilon-Greedy)
26    |       |       |   SE Random(0, 1) < epsilon ENTÃO:
27    |       |       |       |   a <- Escolher_Ação_Aleatória()
28    |       |       |   SENÃO:
29    |       |       |       |   a <- argmax_a' Q[s][a'] (Melhor ação conhecida)
30    |       |       |
31    |       |       |   // Interação com o Ambiente
32    |       |       |   s' <- Calcular_Próximo_Estado(s, a)
33    |       |       |   r <- Avaliar_Recompensa(s')
34    |       |       |
35    |       |       |   SE s' já foi visitado neste episódio ENTÃO:
36    |       |       |       |   r <- r + R_loop // Penalidade de ciclo
37    |       |       |
38    |       |       |   // Atualização de Bellman (Aprendizado Temporal)
39    |       |       |   Max_Q_Futuro <- Máximo(Q[s'] [todas_ações])
40    |       |       |   Q[s][a] <- Q[s][a] + alfa * (r + gama * Max_Q_Futuro - Q[s][a])
41    |       |       |
42    |       |       |   // Transição de Estado
43    |       |       |   s <- s'
44    |       |       |
45    |       |       |   SE s == g ENTÃO:
```

```

46      |   |   |   |   Encerrar Episódio (Objetivo Alcançado)
47      |   |   FIM PARA
48      |   FIM PARA
49
50 3. POS-PROCESSAMENTO E SAÍDA
51      |   Política_Ótima pi(s) <- argmax_a Q[s][a]
52      |   Imprimir Visualização do Grid com pi(s)
53      |   Simular Caminho Final do Agente
54      |   Liberar Memória Alocada
55 -----

```

3. Possibilidade de Paralelização

Com base no código do Q-Learning sequencial, foram identificadas três possíveis abordagens de paralelização:

- Paralelização da obtenção da melhor ação;
- Paralelização dos steps;
- Paralelização dos episódios.

A paralelização da obtenção da melhor ação foi descartada, pois o número de ações disponíveis para o agente é relativamente pequeno, não proporcionando ganho de desempenho significativo que justificasse o custo da paralelização.

A paralelização dos steps também foi descartada, uma vez que cada step depende diretamente do estado atual, e o estado atual é resultado da ação executada no step anterior. Dessa forma, existe uma dependência verdadeira entre as execuções, tornando necessária a sincronização entre as threads e reduzindo os benefícios da paralelização.

Por fim, a paralelização dos episódios foi escolhida como a abordagem mais adequada, pois cada episódio pode ser executado de forma independente. Isso reduz problemas de dependência entre threads e permite melhor aproveitamento do paralelismo durante o treinamento do agente.

4. Maneiras de Paralelizar os Episódios

Como dito anteriormente, a paralelização dos episódios foi escolhida como a mais adequada, porém existe duas possíveis de paralelizar os episódios:

- A primeira maneira é que cada thread vai ter sua Q-Table Local e dado tantos episódios, vai sincronizar com a Q-Table Global (a que será aprendida no final);
- A segunda maneira é que cada thread vai acessar e atualizar diretamente na Q-Table Global. Esse método é chamado de HogWild;

4.1. Com Sincronização

4.2. Hogwild

5. Background and Experimental Context

5.1. Background

5.2. Experimental Context and Workload Details

6. Related Work and Motivation

Check this paper [Schnorr and Legrand 2013] about how you need to semantically aggregate data.

7. Your Great Contribution to Computer Science

8. Your Super Results

9. Conclusion

Acknowledgments

Who paid for this?

References

- Schnorr, L. M. and Legrand, A. (2013). Visualizing more performance data than what fits on your screen. In *Tools for High Performance Computing 2012*, pages 149–162. Springer.
- Schulte, E., Davison, D., Dye, T., and Dominik, C. (2012). A multi-language computing environment for literate programming and reproducible research. *J. of Stat. Soft.*, 46(3).